Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis

Mohammed F. Kharma, Soohyeon Choi, Mohammad Alkhanafseh, and David Mohaisen, *Senior Member, IEEE*

***Abstract*—Artificial Intelligence (AI) driven code generation tools are increasingly used throughout the software development lifecycle to accelerate coding tasks. However, the security of AI-generated code using large language models (LLMs) remains underexplored, and recent studies have revealed various risks and weaknesses. This paper presents a measurement study of LLM-generated code across four programming languages (Python, Java, C++, and C) and five widely used LLM families. We construct a manually curated dataset of 200 programming tasks, grouped into seven functional and security-relevant categories, each with language-neutral specifications. For every combination of task, language, and model, we generate code and evaluate it along three axes: syntactic validity and compilation success, semantic correctness using 4,000 per program unit test files, and software quality and security using SonarQube and CodeQL, complemented by manual review of key static analysis findings. Our results show clear language effects: Python and Java achieve higher compilation and semantic correctness rates and produce fewer security findings than C and C++, where we observe more memory safety issues, hard-coded secrets, and cryptographic misuses. We also find that many models fail to make use of modern security features available in recent compiler and toolkit updates (i.e., in Java 17), and that outdated methods remain common, particularly in C++. These findings highlight the need to advance LLMs so that they better align with emerging secure coding practices and language-specific best practices.**

***Index Terms*—LLM, AI-generated code, security, measurement.**

I. Introduction

THE rapid advancement of artificial intelligence (AI) technologies has driven the widespread adoption of large language models (LLMs) for generating source code across a broad range of programming tasks. In particular, LLMs have demonstrated remarkable performance in natural language understanding and generation. Building on these capabilities, recent LLMs have emerged as powerful tools for software developers, enabling the automated generation of functional code in multiple programming languages.

Received 10 September 2025; revised 2 March 2026; accepted 8 March 2026. Date of publication 10 March 2026; date of current version 12 May 2026. *(Corresponding author: David Mohaisen.)*

Mohammed F. Kharma and Mohammad Alkhanafseh are with the Department of Computer Science, Birzeit University, Ramallah 627, Palestine (e-mail: mkharmah@birzeit.edu; malkhanafseh@birzeit.edu).

Soohyeon Choi and David Mohaisen are with the Department of Computer Science, University of Central Florida, Orlando, FL 32816 USA (e-mail: soohyeon.choi@ucf.edu; mohaisen@ucf.edu).

All code and data are available at GitHub.

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2026.3672745, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3672745

In 2018, Microsoft Visual Studio released the IntelliCode extension [1], which offers AI-powered development features that provide limited insights by analyzing code context using machine learning. In 2021, GitHub introduced Copilot, an AI-driven code assistant designed to improve coding quality by training on extensive real-world code repositories [2], enabling it to provide coding recommendations across various programming languages and frameworks. Since GitHub introduced Copilot, AI-driven code generation adoption has grown in the software development lifecycle [3], where complex models are used to perform specific tasks like writing code for software engineers [4].

One major challenge for AI-driven code generation is ensuring code quality, where key metrics include validity, correctness, security, reliability, and maintainability. Current research stresses the importance of ensuring security in LLM-generated code [5], [6]. Several other studies have shown that LLMs are capable of producing code that is both secure and vulnerable [4], [5], [7]. The factors affecting the security of the code generated by these models are not identified. Multiple studies highlight the need for better user guidelines and awareness when interacting with AI tools [5], [6], [8] to enhance the quality properties of generated code. The security quality of the code generated for identical scenarios may differ depending on the chosen programming language, which is considered an explainable issue [6], [9].

This work addresses this gap by investigating the factors that impact the security of LLMs-generated code, with a focus on the influence of language selection. Our contributions include the development of a dataset designed to evaluate AI-generated code across multiple domains, such as problem-solving, algorithms, and other key areas. Additionally, we evaluate the ability of various LLMs (*claude-3.5*, *gemini-1.5*, *codestral*, *GPT-4o*, *llama-3*) to generate secure and functional code in different languages (Python, Java, C++, and C). Moreover, We conduct a quality and security analysis of LLM-generated code using static security analysis tools (SAST) to identify common vulnerabilities and weaknesses, and quantify their distribution across models and languages, focusing on how the known CWE categories manifest in a unified evaluation setting. Lastly, this research conducts a comparative study of the security properties of AI-generated code across different programming languages, highlighting the main strengths and limitations of each AI tool in contexts such as semantic correctness and security.

Research Questions: We address this problem through the following research questions. *RQ1:* How does the choice of

1545-5971 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

programming language affect the security of code generated by large language models (LLMs)? *RQ2:* How do different LLMs compare in generating syntactically valid and semantically correct code across multiple programming languages? *RQ3:* Which security vulnerabilities are most prevalent in LLM-generated code, and how do they vary across models and programming languages?

Contributions: We make the following contributions. (1) *New dataset for LLM-based coding:* We introduce a new manually vetted dataset of 200 programming tasks from seven categories that can be used by the research community for evaluating the performance of LLMs. (2) *Comparative evaluation of LLM-generated code.* We evaluate the quality and security of LLM-generated code across multiple programming languages and models using a unified dataset and a consistent evaluation setup. This enables direct cross-language and cross-model comparisons and exposes differences that are missed in single-language or single-model studies. We treat code quality and security as complementary dimensions, as low-quality code often correlates with security vulnerabilities and secure coding practices align with readability and maintainability. Joint evaluation establishes a baseline for analyzing their interactions and trade-offs in LLM-generated code. (3) *In-depth security analysis.* We analyze LLM-generated code across multiple programming languages to identify language-specific characteristics, recurring issues, and distinct security vulnerability patterns.

Organization: We provide a review of related work in Section II, the research methodology is described in Section III, the proposed dataset is presented in Section IV, analysis results and discussion in Section V, and the concluding remarks and future work in Section VII.

II. Related Work

Several works explored LLMs's use in a variety of software development activities [3], [4], [5], [6], [7], [8], [11], [12], [14], [17], [20], [21], [22], [24], [25], [26], [27]. This section examines key studies, emphasizing their methods, applications, and the analyzed features. The comparative overview helps place our work in the broader research context.

Huang et al. [26] surveyed pre-trained models and LLMs for software engineering tasks, including requirements, code, test case, patch generation, optimization, summarization, and code translation, covering models such as *BERT*, general transformers, and *ChatGPT*. Our work builds on this line of research by focusing specifically on code generation and evaluating the effectiveness of LLMs in this setting. Perry et al. [5] analyzed the security of LLM-generated code and showed that it often contains more security flaws than human-written code, with LLMs exhibiting overconfidence in code safety. Sandoval et al. [11] reported a 10% increase in vulnerabilities in LLM-generated C code. Asare et al. [3] found that *GitHub Copilot* can improve security for complex tasks but provides limited benefit for simpler ones. In contrast to prior work, we evaluate four LLMs across multiple programming languages, including C++ and Java, to enable broader cross-language and cross-model analysis.

Yetiştiren et al. [14] evaluated the quality of code generated by three code assistants using the HumanEval benchmark [15]. They reported correctness rates of 65.2%, 46.3%, and 31.1% for *ChatGPT*, *Copilot*, and *CodeWhisperer*, respectively, and analyzed the impact of function names, inputs, and problem descriptions. Security, maintainability, and reliability were assessed using SonarQube [28], with all tools producing code deemed secure. Our work differs by evaluating four programming languages and using a broader and more diverse set of dataset scenarios.

Asare et al. [12] compared *GitHub Copilot's* code generation to human-written code for security vulnerabilities, using a dataset by Fan et al. [13]. They found that *Copilot* recreated the same vulnerabilities in 33.3% of cases and remedied 25.5% of them. *Copilot* showed inconsistencies, especially with older vulnerabilities, but generally produced fewer security flaws than humans. Khoury et al. [6] tested *ChatGPT's* ability to generate secure code in various languages, showing that it often failed to meet security standards but improved with follow-up prompts. *ChatGPT* corrected 12 of the 21 programs when prompted.

Nair et al. [8] explored *ChatGPT's* effectiveness in generating hardware code using Common Vulnerability Enumerations (CWE-1194), showing the possibility of guiding AI to avoid common flaws. Elgedawy et al. [4] analyzed *GPT-3.5*, *GPT-4*, *Bard*, and *Gemini*, showing that using security personas reduced vulnerabilities, especially in *GPT-3.5*, *GPT-4*, and *Bard*. Siddiq et al. [7] introduced the SALLMS framework to evaluate LLMs systematically for security. Schuster et al. [20] and Wu et al. [17] highlighted challenges, such as data poisoning and the reduced effectiveness of LLMs in handling complex security issues, highlighting the need for stronger defenses and better vulnerability detection.

Ullah et al. [21] evaluated modern LLMs for vulnerability detection and identified limitations in consistency, reasoning accuracy, and robustness, particularly under minor code changes and real-world conditions. Pearce et al. [27] analyzed the security implications of code generated by *GitHub Copilot* and showed that the tool frequently produces vulnerable patterns, especially when prompts are underspecified or ambiguous. Siddiq and Santos [22] introduced SecurityEval, a dataset of 130 Python programs mapped to 75 common weakness enumerations (CWEs), to evaluate the security of machine-generated code. Using SecurityEval, they assessed tools such as *GitHub Copilot* and *InCoder* and found that these models often generate code with security vulnerabilities, emphasizing the need to evaluate generated code beyond functional correctness to include security.

Asare et al. [12] evaluated the ability of LLMs to detect and reason about security vulnerabilities in source code across multiple languages and tasks. The study showed that most LLMs fail to identify deeper security flaws, even when the code is syntactically correct, highlighting the need for more reliable, security-focused benchmarks and evaluation methods. Tony et al. [24] introduced LLMSecEval, a dataset of natural language prompts designed to assess LLM performance on security-related questions across common software security topics, enabling consistent evaluation of security understanding. Together, these studies stress the importance of evaluating LLMs beyond code generation, with explicit focus on security reasoning.

Lenarduzzi et al. [25] compared six code analysis tools and evaluated their issue detection effectiveness, result consistency,

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I
A SUMMARY OF THE RELATED WORK. HIGHLIGHTED THE EVALUATED PROGRAMMING LANGUAGES AND THE LLMS. LANGUAGES: ① *C*, ② *C++*, ③ *JAVA*, ④ *PYTHON*, ⑤ *JAVASCRIPT*, ⑥ *HTML*, ⑦ *VERILOG*. LLMS: ⑧ *COPILOT*, ⑨ *CODEX*, ⑩ *WHISPER*, ❶ *GEMINI*, ❷ *BARD*, ❸ *GPTS*, ❹ *LLAMA-3*, ❺ *CLAUDE-3.5*, ❻ *CODESTRAL*, ❼ *STARCODER*, ❽ *CODEGEN*.

Reference	Year	Programming Languages							Large Language Models									Prompt Scenario
		①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	❶	❷	❸	❹	❺	❻	❼	❽	
Asare *et al.* [3]	2024	✔							✔											Two problems [10]
Perry *et al.* [5]	2023	✔			✔	✔				✔										Six tasks [5]
Sandoval *et al.* [11]	2023	✔								✔										Shopping list function
Asare *et al.* [12]	2023	✔	✔						✔											Big-Vul dataset [13]
Yetistiren *et al.* [14]	2023				✔				✔		✔			✔						HumanEval dataset [15]
Khoury *et al.* [6]	2023	✔	✔	✔	✔		✔		✔											21 tasks [16]
Nair *et al.* [8]	2023							✔	✔											Scenarios from the selected CWEs
Elgedawy *et al.* [4]	2023				✔							✔	✔	✔						Nine tasks
Wu *et al.* [17]	2023			✔	✔									✔						SARD and Juliet datasets [18]
Siddiq *et al.* [7]	2023				✔									✔				✔	✔	LLMSecEval dataset [19]
Schuster *et al.* [20]	2021				✔									✔						-
Ullah *et al.* [21]	2024	✔	✔	✔	✔										✔		✔	✔	✔	200 tasks
Siddiq *et al.* [22]	2022				✔				✔											SecurityEval dataset [19]
Khare *et al.* [23]	2024	✔	✔	✔					✔		✔	✔		✔						25 CWEs, dataflow based prompt
Asare *et al.* [?]	2024	✔	✔	✔	✔									✔		✔		✔		200 Tasks
Tony *et al.* [24]	2023	✔			✔							✔				✔		✔		LLMSecEval dataset [19]
Lenarduzzi *et al.* [25]	2021			✔	✔							✔								LLMSecEval dataset [19]
This work	2024	✔	✔	✔	✔							✔		✔	✔	✔	✔			200 tasks

TABLE II
SUMMARY OF RELATED WORK. OVERALL IMPACT (OI): NEGATIVE (N), POSITIVE (P), AND NEGATIVE DUE TO THE USE OF AN INAPPROPRIATE DATASET FOR SECURITY (N*). ATTRIBUTES: VALIDITY, CORRECTNESS, SECURITY, RELIABILITY, AND MAINTAINABILITY. METHODS: MANUAL, STATIC SCAN, AND RUNTIME SCAN.

Reference	Quality Attributes						Analysis Method			OI
	V	C	Se	Re	Ma	Pe	M	S	R	
Asare *et al.* [3]			✔				✔			N
Perry *et al.* [5]			✔				✔			N
Sandoval *et al.* [11]			✔				✔	✔	✔	N
Asare *et al.* [12]			✔				✔	✔		P
Yetistiren *et al.* [14]	✔	✔	✔	✔	✔		✔	✔		N*
Khoury *et al.* [6]			✔				✔			N
Nair *et al.* [8]			✔				✔			P
Elgedawy *et al.* [4]	✔	✔	✔	✔		✔	✔	✔		N
Wu *et al.* [17]			✔				✔			N
Siddiq *et al.* [7]			✔					✔		N
Khare *et al.* [23]	✔	✔	✔		✔	✔		✔		P
Asare *et al.* [3]			✔				✔			N
Tony *et al.* [24]			✔					✔		N
Lenarduzzi *et al.* [25]			✔					✔		N
Ullah *et al.* [21]			✔					✔		N
Siddiq *et al.* [7]			✔					✔		N
Schuster *et al.* [20]			✔				✔			N
This work	✔	✔	✔	✔	✔		✔	✔		N

and precision and reported substantial differences in detection coverage and overlap among tools, showing that reliance on a single tool can miss vulnerabilities or introduce false positives. Khare et al. [23] performed a large-scale evaluation of LLM-based vulnerability detection, testing 16 models on 5,000 code samples from five datasets of C, C++, and Java. The authors proposed dataflow-inspired prompting strategies and showed that LLMs can outperform static and deep learning-based tools for specific vulnerability classes while providing interpretable explanations.

Table I summarizes prior work by language, LLM, and prompt for LLM-generated code. Table II categorizes related work by the quality attributes examined and the security analysis methods applied to evaluate generated code, and reports the overall impact of AI code generation on code quality based on each study's experimental results.

Our Work: Most prior studies analyze LLM code generation behavior but provide limited insight into the factors affecting code quality and security beyond user demographics and experience levels. This work examines how programming language characteristics influence the quality of LLM-generated code.

TABLE III
LLMS USED FOR EVALUATION AND THEIR SHORT NAMES

Provider	Ref	Model Version	Short
OpenAI	[29]	GPT-4o-202407	*GPT-4o*
Perplexity	[30]	llama-3-sonar-large-32k-chat	*llama-3*
Anthropic	[31]	claude-3-5-sonnet-20240620	*claude-3.5*
Mistral AI	[32]	codestral-2405	*codestral*
Google	[33]	gemini-1.5-pro-001	*gemini-1.5*

We evaluate validity, correctness, security, maintainability, consistency, intentionality, adaptability, and responsibility across four languages and five LLMs. This study is a quantitative measurement study: it introduces no new vulnerability classes or analysis techniques, but systematically compares the behavior of existing LLMs across languages and tasks under a unified setup.

III. METHODOLOGY

We selected LLMs using a two-step process. First, we defined eligibility criteria based on prior literature and industry benchmarks: ① widespread adoption in software engineering practice, ② demonstrated performance in program synthesis, reasoning, and security-related tasks, ③ official support for code generation in Python, Java, C++, and C, ④ stable and accessible APIs during data collection, and ⑤ architectural diversity to avoid evaluating closely related model variants. Second, we applied these criteria to available models and list the selected LLMs in Table III. The selected models are widely used, frequently evaluated in prior studies, support all target languages through stable APIs, and represent distinct training and design choices, enabling systematic comparative analysis.

As is known, the internal architecture and training methodologies of these LLMs, i.e., can introduce significant variation in the code that is produced. This study aims to provide information on how these variations influence the security posture of generated code, focusing on the importance of selecting the right LLM based on specific development needs. Table IV highlights the context window and other configurations used when generating code using each model. Max_tokens refers to the maximum number of tokens to generate in completion. Top_p changes how the model selects tokens for output. Tokens are selected from the

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
LLMs, TEMPERATURE (TEMP), MAXIMUM TOKENS (MAXT), CONTEXT WINDOW (CW), AND TOP_P (TOPP)

Model	Temp	MaxT	TopP	CW
GPT-4o	0.9	4,096	0.9	128k
llama-3	0.9	4,096	0.9	32k
claude-3.5	0.9	4,096	0.9	200k
codestral	0.9	4,096	0.9	32k
gemini-1.5	0.9	4,096	0.9	128k

most probable to least until the sum of their probabilities equals the top-p value. The model temperature is used to control the randomness in generating the output. The context window is the maximum token count a model can handle in one forward pass, covering both the input (prompt) and the output. It essentially determines the amount of text the model can process at a time.

Temperature Setting: As shown Table IV, all experiments were conducted with temperature $T = 0.9$. This value was selected to reflect practical code-generation usage, where moderate sampling diversity is desirable while maintaining output stability. A higher temperature encourages exploration of alternative implementations, which is relevant for assessing robustness and variability in generated code. To evaluate the sensitivity of our results to this parameter, we performed a supplementary ablation using a lower temperature setting. The analysis shows that relative model rankings remain consistent, cross-language performance trends are preserved, and the primary conclusions regarding semantic correctness, security findings, and quality metrics are unchanged. These observations indicate that our findings are robust to reasonable variations in temperature and are not an artifact of the chosen sampling parameter.

A. Programming Languages Selection

One of the key motivations and contributions of this study is the comparative exploration of the performance and security of LLMs under the same evaluation settings for different programming languages. We determine a range of programming languages to be evaluated in our evaluation, covering both statically and dynamically typed languages. As such, we choose the following programming languages as a preliminary set that meets those metrics: ① C; ② C++; ③ Java; ④ and Python. Although our choice of programming language is limited by the capabilities of LLMs and the programming languages they support, we believe that these programming languages are representative, so they are among the top five most used programming languages [34].

Each programming language exhibits attributes that affect security outcomes in code generation (e.g., Python's dynamic typing versus C++'s static typing leads to different classes of bugs and vulnerabilities). Java and Python provide automatic memory management, reducing memory-related errors relative to C and C++, which rely on manual memory management. These languages therefore enable analysis of how LLMs handle language-specific security properties. The selection also improves external validity, as these languages rank among the top five most used in 2024 and reflect common real-world development settings.

B. Dataset

To comprehensively evaluate LLM performance, we curated 200 prompt descriptions targeting diverse aspects of code generation. Task selection ensured broad coverage of core programming paradigms and secure coding practices. Details are provided in Section IV.

C. Environment Setup

To ensure consistency and reproducibility, we used a standardized environment for code generation, compilation, execution, and validation across tasks and programming languages, including unit test support. All experiments ran on a Lenovo ThinkPad E570 with a 7th-generation Intel Core i7 processor, 16 GB DDR4 RAM, and a 256 GB SSD. This configuration provides sufficient performance and portability for LLM integration, code generation, and multi-language compilation. We used Debian 12 for its stability, package management, and low resource overhead, ensuring a consistent platform for cross-language testing. The following software packages and versions were used. ❶ **Java.** We compiled and executed Java code using OpenJDK 17.0.8 (LTS), which supports modern Java language features and ensures compatibility with constructs generated by LLMs. ❷ **Python.** We evaluated Python code using Python 3.11.9 to ensure compatibility with current features and libraries. ❸ **C and C++.** We compiled C and C++ code using CMake 3.28.6. For C++, we set `CMAKE_CXX_STANDARD` to 17, a widely adopted standard since its 2017 release [35], ensuring support for modern C++ features produced by LLMs.

This setup offered a stable basis for assessing the code produced by the five chosen LLMs. By preserving identical hardware and software settings, we ensured that any differences in code compilation, run-time, or accuracy between programming languages or models are due to the models themselves and not to environmental factors.

D. LLM Integration and Code Generation

Based on the LLMs listed in Table III, we generated code for each task in four programming languages. A custom Python script sequentially dispatched task prompts to each model and collected the generated code. All inputs used the original task descriptions without prompt engineering. After generation, a second Python script parsed and organized the outputs into a structured file system, assigning language-specific file extensions (e.g., `.py` for Python and `.java` for Java). Each file name encoded the task identifier, LLM, and programming language to support traceability and comparison. The first author verified the total number of generated programs to ensure complete collection.

As a result, 4,000 code files were generated, comprising 200 files per language for each LLM. This dataset formed the foundation for the subsequent evaluation procedures, ensuring that each LLM was evaluated using an identical set of tasks and programming languages.

E. Quality Evaluation

To evaluate code quality, we analyze syntax validity, functional correctness, code length, reliability, maintainability, and

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

security [7], [14] using two evaluation approaches. ① *Manual evaluation* by human experts where two software engineers with four and ten years of professional experience participated in the evaluation, with the first and second authors coordinating and reviewing the process. Drafting, validating, and executing test cases to assess semantic correctness required over 800 person-hours across five months. ② *Automated static analysis* using SonarQube [28] and CodeQL [36], which support all target languages. Automated tools provide scalable coverage of quality and security metrics, but have known limitations when used in isolation. Human evaluation remains the gold standard for assessing LLM outputs. We therefore complement automated analysis with human evaluation on a representative subset of generated programs to support generalization beyond a limited number of evaluators. Prior work identifies these methods as effective for code quality and security assessment, supporting our choice of metrics and tools.

1) Semantic Soundness:

Compilation-Time Errors: Before evaluating functionality and quality, we first ensured the code met syntactic standards and compiled without errors. This step was crucial for valid and reliable assessments.

For each programming language, automated scripts were created to perform syntax checks and were reviewed by one of the software engineers and the second author. For compiled languages (C, C++, and Java), each source file was compiled and the outcome recorded. For interpreted languages like Python, syntax validation was conducted using Python's built-in checker. Results were documented in a matrix of 200 rows (tasks) by 20 columns (5 LLMs $\times$ 4 languages), where each cell contained a binary value indicating syntactic validity or successful compilation. This process enabled early identification of issues such as syntax errors or missing imports, allowing for the correction or exclusion of problematic samples prior to semantic analysis.

Semantic Correctness Using Unit Tests: We evaluated semantic correctness after resolving all syntactic errors by checking whether each generated program implemented the intended logic specified in the task prompt.

The initial goal was to create a single reusable unit test per task and language. Natural language prompts without fixed function signatures led to substantial variation in generated code, including differences in function names, parameters, return values, and I/O handling. This variability reflects realistic LLM use but prevented uniform test reuse.

We addressed this variability using a four-phase unit testing procedure. First, for each of the 200 tasks, we defined a language-agnostic specification describing intended behavior, input domains, and test scenarios, including normal, boundary, edge, and invalid cases. Reference implementations produced expected outputs. Second, after code generation, we analyzed each program's interface, including function signatures, parameter types and order, output mechanisms, and data structures (e.g., Python lists versus `std::vector` in C++), which ruled out a unified test suite.

Third, we implemented per-sample unit tests tailored to each program's structure while preserving the shared logical specification. All tests used the same inputs and expected outputs across languages and models, adapting only syntax and data types. Assertions targeted return values or captured standard output based on program behavior. Fourth, two professionals manually reviewed each test for correctness, and the first and second authors re-checked a representative subset across languages and task categories. This process enabled consistent and fair evaluation of semantic correctness despite structural differences in generated code.

In total, we created 4,000 unit test files, corresponding to 200 tests per programming language per LLM. Unit test outcomes were recorded in a matrix with the same dimensions used for compilation-time error analysis. Each unit test received a score from 0% (no tests passed) to 100% (all tests passed), based on the proportion of successful test cases. Two developers executed the testing, and the first and second authors conducted a single review cycle of the unit test files. This process quantified semantic correctness and reflected how accurately LLMs interpreted task specifications. The evaluation posed challenges, including inconsistent function signatures across LLM outputs, which prevented standardized testing. We addressed this by creating per-sample unit tests, increasing implementation effort but ensuring correctness.

Manual Fix Protocol: During preprocessing, a small subset of generated files required minor manual adjustments to enable compilation. In total, 73 out of 4,000 generated programs (1.8%) were modified across all languages and models. These modifications were strictly limited to compilation-enabling corrections, including: (i) adding missing import or include statements, (ii) correcting trivial syntax errors such as missing semicolons or unmatched braces, (iii) resolving formatting artifacts introduced during model output (e.g., truncated code blocks), and (iv) adjusting file-level declarations required by the build environment.

No modifications affected control flow, logic, data structures, or security-relevant operations, and no vulnerability patterns were altered or removed. All metrics (compilation, semantic correctness, and static analysis) were computed under a uniform evaluation protocol across all models and languages; the fixes were strictly syntactic and did not alter program functionality or security characteristics.

Beyond semantics, we analyzed code coverage on a representative subset of tasks using JaCoCo [37] for Java and comparable tools for other languages. Each task included up to 10 unit tests covering standard cases and edge conditions. Coverage analysis complemented correctness checks by assessing test thoroughness. Average line coverage across sampled tasks exceeded 75%, indicating that most logic paths in the generated code were exercised. This coverage level supports the robustness of test design and evaluation.

2) Complexity: Understanding code complexity is essential for assessing program maintainability and readability. Higher complexity increases the likelihood of bugs, complicates testing, and lowers developer productivity. In this work, we analyze established complexity metrics to capture various structural and cognitive aspects of difficulty in LLM-generated code.

① *Lines of Code (LoC)* measures the program's lines of code, excluding whitespace. LoC is a predictive metric used to evaluate effort and maintainability. ② *Cyclomatic Complexity (CyC)*

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

calculates code complexity using a control flow graph (CFG). With E as edges, N as nodes, and Q as connected components, CyC is computed as $M = E + 2Q - N$ [38]. ③ *Cyclomatic Complexity Density (CCD)* measures how cyclomatic complexity spreads across the codebase. With cl as the total code lines, CCD is calculated as $Md = (E + 2Q - N)/cl$ [39]. ④ *Cognitive Complexity (CoC)* measures the difficulty of understanding code [40]. CoC considers structure like control flow and nesting, using $C = C_{\text{base}} + \sum_{i=1}^{n} nc$, where nc represents increases due to nesting and conditionals.

3) Quality and Security: We evaluate software quality and security using SonarQube [28] and CodeQL [36], two widely adopted static analyzers supporting all target languages. SonarQube is used to assess non-vulnerability quality dimensions, while CodeQL provides query-based validation of vulnerabilities. Consistent with prior work, we manually reviewed all reported vulnerabilities and validated a stratified 20% sample across languages and LLMs to mitigate false positives and contextual limitations. Our evaluation covers four indicators: reliability, security (vulnerabilities), maintainability, and security hotspots. In total, the analysis spans 97,412 lines of code across four languages by five LLMs. ① *Reliability* assesses correct operation under predefined conditions using reported bug findings. ② *Security* captures exploitable weaknesses, quantified by the number and severity of detected vulnerabilities. ③ *Maintainability* measures ease of understanding and modification based on code smells and complexity indicators. ④ *Security Hotspots* identify security-sensitive code regions requiring careful handling, such as authentication, authorization, and input validation.

4) Clean Code: We evaluate clean code in LLM-generated outputs using four dimensions derived from SonarQube's static analysis: consistency, intentionality, adaptability, and responsibility.

① *Consistency* assesses formatting, naming conventions, and structural uniformity across languages and prompts, including spacing, indentation, and identifier casing. Consistent code improves readability and reduces review and maintenance overhead. SonarQube captures consistency issues via code smells such as improper naming, unused imports or dead code, and inconsistent indentation or braces, which are aggregated under maintainability due to their impact on code clarity.

② *Intentionality* evaluates whether the code clearly and efficiently implements the intended functionality without unnecessary complexity. SonarQube captures this attribute through cognitive complexity metrics and code smells that penalize deep nesting, large switch blocks, unclear logic, and bloated methods, directly contributing to maintainability.

③ *Adaptability* measures ease of modification through modular design and separation of concerns, enabling localized changes with minimal risk. SonarQube assesses adaptability using function length, parameter count, and lack of modular decomposition (e.g., monolithic blocks), which affect maintainability and may trigger security hotspots when sensitive logic lacks encapsulation.

④ *Responsibility* reflects compliance with professional and ethical standards, including secure data handling and avoidance of high-risk practices. We evaluate responsibility through detected vulnerabilities (e.g., hard-coded credentials), security hotspots (e.g., unchecked inputs or return values), and use of deprecated or insecure APIs, spanning both security and maintainability dimensions.

Analysis: We analyze the results across key quality attributes, including syntactic validity, semantic correctness, security, reliability, maintainability, and clean code properties. Detailed results appear in Section V.

To improve methodological transparency, Fig. 2 (appendix F, available online) summarizes the complete Quality Control (QC) pipeline applied to each task, covering dataset design, code generation, compilation, semantic testing, and static analysis.

IV. Dataset

To advance the evaluation of the quality and security of LLM code generation, we developed a dataset that reflects practical and security-relevant challenges in real-world programming domains. Compared to previous work, the dataset offers several advantages: it is created manually, language-independent, and tailored to evaluate LLM-generated code across multiple dimensions, including code correctness, maintainability, reliability, and security. Our goal is to provide the research community with a robust foundation for testing, comparing, and improving LLMs in terms of functional correctness, code quality, and security-oriented scenarios by integrating tasks rooted in known vulnerabilities (e.g., CWEs) and actual programming practices, thereby enhancing their overall utility.

What distinguishes our dataset is both its breadth, spanning 200 tasks across diverse categories, and its depth, with each task annotated using expert-reviewed tags and accompanied by unit tests designed to evaluate LLM-generated code. The dataset is model-agnostic, supports four programming languages, and adheres to secure coding best practices. Thus, it functions as benchmarking resource and a flexible training and diagnostic tool for researchers and practitioners engaged in LLM-assisted code development.

A. Dataset Construction

Our dataset construction methodology consists of four major steps: functional categorization of tasks, task sources and cross-language adaptation, task validation and peer review, and task tagging and redundancy reduction. In the following, we review those steps in detail.

Functional Categorization: We first organized the tasks into functional categories, reflecting both foundational programming skills and real-world software security concerns:

① *Secure Coding:* tasks grounded in known software weaknesses based on MITRE's CWE taxonomy, including issues such as injection flaws, cryptographic misuse, and insecure data handling;

② *Data Structures and Algorithms:* problems focusing on core computing concepts such as array manipulation, recursion, sorting, searching, and graph traversal;

③ *Parsing and Validation:* tasks requiring careful input processing, constraint enforcement, and boundary checking to assess robustness and correctness;

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE V
TASK DISTRIBUTION BY SOURCE AND DIFFICULTY (E: EASY, M: MEDIUM, H: HARD)

Source	Description	E	M	H	Total
Manual	Custom tasks written by the first author of this research	3	117	3	123
Code challenge	Codewars [43], LeetCode [41], Exercism [49], and Edabit [42]	14	43	7	64
Security dataset	SecurityEval [22], CodeQL [50]	0	13	0	13

④ *Networking:* scenarios that emulate networked systems, including authentication errors and protocol misuse;
⑤ *Mathematics and Logic:* challenges designed to evaluate numerical reasoning, precision, and logical flow;
⑥ *Programming Systems and Utilities:* utility tasks involving file handling, scripting, system configuration, and general-purpose development workflows;
⑦ *Concurrency and Synchronization:* tasks targeting race conditions, thread safety, and synchronization issues in parallel execution contexts.

Task Sources and Cross-Language Adaptation: We sourced tasks from coding challenge platforms, including LeetCode [41], Edabit [42], and Codewars [43], as well as from vulnerability databases [44] and academic datasets [22]. When suitable tasks were unavailable, we designed additional tasks based on realistic software engineering scenarios. We formulated each task in neutral language to support consistent implementation in Python, Java, C++, and C, enabling fair cross-language comparison. Tasks originally defined for a single language, such as Python tasks from SecurityEval, were rewritten for cross-language applicability without changing core logic, intent, or security focus.

Our design choice of concise, self-contained programming tasks, adopted partially from popular coding challenge platforms, is consistent with recent large-scale evaluations of LLM-generated code on tasks from such sources, where problems with clear specifications and automated tests are used to enable systematic comparison across models [45], [46], [47], [48]. In contrast to these works, our benchmark additionally incorporates security-focused tasks and supports four programming languages, while preserving the same style of well-specified, unit-testable problems. Table V summarizes the distribution of tasks by source and categorizes them by difficulty level. Only a small subset of the 200 is labeled *Easy*, while the large majority fall into the *Medium* or *Hard* categories. This distribution is intentional and reflects our focus on non-trivial problems that require reasoning about real-world cases, input validation, and security-relevant behavior rather than purely direct tasks.

Task Validation and Peer Review Process: To ensure the quality, clarity, and reliability of each task in the dataset, we implemented a structured peer review process involving two (professional) software engineers working alongside the second author of this study. The reviewers systematically assessed each task for clarity, difficulty level, and solvability. Their feedback was discussed and incorporated through multiple review cycles, with any ambiguities, inconsistencies, or edge cases resolved collaboratively.

Unit test is used to verify task correctness, where we established a unified unit test specification applied consistently across all studied programming languages. This specification defined identical inputs, expected outputs, and coverage scenarios, with only language-specific syntax that varied between implementations. An illustrative example of these test scenarios is provided in Appendix E, available online. This unified method ensures that variations in results are attributable to differences in code generation quality rather than inconsistencies in validation, thus providing a fair and controlled basis for comparing LLM-generated codes across languages.

Task Tagging and Redundancy Reduction: Each task was manually reviewed and tagged by at least two reviewers according to its functionality and any implicit security objectives. Tasks covering multiple concepts were assigned multiple tags. This approach enables the evaluation of LLM performance on both narrowly focused and multi-dimensional tasks. To avoid semantic redundancy, we conducted a manual similarity analysis across task prompts and implementations, removing tasks that exhibited significant overlap in logic or structure. In future work, we plan to incorporate automated semantic similarity techniques to further validate dataset diversity and reduce potential bias.

B. Dataset Features and Improvements

The dataset consists of several attributes to guide the evaluation, including: ① task number, which refers to unique identifier assigned to each question; ② prompt title, which refers to brief title that summarizes the problem statement; ③ description, a detailed description of the task, outlining the problem to be solved; ④ hints, which refers to instructions that provided to AI model to guide the generation of the code; ⑤ solutions, represent the code generated by the LLM for each programming language along with the name of the model used; ⑥ source, which refers to the origin of the task, whether manually created or adapted; ⑦ test cases, which refers to 10 test cases per prompt were written to evaluate the semantic of the generated code; ⑧ tags; labels assigned to each task to facilitate filtrating, categorization, and statistical analysis; ⑨ comments, which refers to notes made by reviewers to document the evaluation process, including issues encountered during the generation process.

To improve interpretability and enable more focused evaluation, we consolidated the original set of task tags into a structured set of seven semantic categories. Tags were initially assigned through manual annotation based on detailed task analysis; however, as the dataset expanded, the tag set exhibited redundancy, inconsistent phrasing, and varying levels of specificity. To address this, we applied a *semantic grouping* strategy by clustering related tags under broader functional themes. For instance, technical concepts such as *arrays*, *sorting*, *graphs*, and *recursion* were unified under **Data Structures and Algorithms**, reflecting core computational topics. Likewise, all tags referencing Common Weakness Enumeration (CWE) identifiers, including *CWE-20*, *CWE-89*, and *CWE-787*, were categorized under **Secure Coding (CWEs)**, encompassing tasks focused on recognizing and addressing software vulnerabilities. Table VI shows how tasks are distributed according to tag categories, achieved by combining similar tags into larger functional groups. A task may belong to one or several categories.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
DISTRIBUTION OF TASKS BY TAG CATEGORIES AFTER GROUPING RELATED TAGS INTO BROADER FUNCTIONAL AREAS. ONE TASK MIGHT FALL INTO ONE OR MULTIPLE CATEGORIES.

Category	Task Count
Secure Coding (CWEs - All CWE identifiers grouped)	102
Data Structures and Algorithms	86
Parsing and Validation	54
Networking	48
Mathematics and Logic	26
Programming Systems and Utilities	18
Concurrency and Synchronization	6

TABLE VII
MITRE-ALIGNED CATEGORIZATION OF REPRESENTATIVE CWES IN THE DATASET

CWE Category	Representative CWEs
Input Validation	CWE-20, CWE-707, CWE-642
Injection	CWE-89, CWE-78, CWE-74
Buffer Errors	CWE-119, CWE-787, CWE-125
Access Control Issues	CWE-285, CWE-639, CWE-287
Cryptographic Issues	CWE-321, CWE-326, CWE-327
Information Exposure	CWE-209, CWE-532
Resource Management Errors	CWE-400, CWE-770
Memory Errors	CWE-416

To illustrate dataset structure and content, Appendix B, available online presents representative tasks from each programming category. These examples reflect the range of real-world scenarios in the benchmark and contextualize the evaluation by showing the scope and complexity of the prompts. We release the complete dataset as part of the replication package, including task descriptions, difficulty labels, unit tests, and model outputs, enabling detailed inspection and reproduction or extension of the evaluation.

C. Secure Coding Coverage Via CWE Grouping

To assess model behavior on security-critical tasks, we integrated 86 unique CWE identifiers into our dataset, as noted in the task tagging. Rather than presenting these individually, we grouped them into higher-level vulnerability domains based on their semantic alignment and the MITRE taxonomy. These categories include Input Validation (i.e., CWE-20, CWE-707), Injection (i.e.,, CWE-89, CWE-78), Buffer Errors (i.e., CWE-119, CWE-787), Access Control Issues, Cryptographic Issues, and others. This organization provides a clearer view of the dataset's security coverage and enables targeted evaluation of model resilience against specific classes of vulnerabilities. Table VII presents the distribution of tasks across these security domains, and the full list of CWEs is available in our project repository.

V. RESULTS AND DISCUSSION

We analyze the performance of different LLMs with respect to code generation, including correctness, security, and reliability. The evaluation process depends on different metrics, i.e., compilation-time errors, security, and overall accuracy across different languages. Each LLM's output is compared to define the best generation tool.

TABLE VIII
BREAKDOWN (%) OF LLM-GENERATED CODE FILES WITHOUT COMPILATION-TIME ERRORS (C) AND SEMANTIC ISSUES (S)

Model	Java		Python		C++		C	
	C	S	C	S	C	S	C	S
claude-3.5	95.0	91.8	99.5	92.1	81.5	91.2	96.0	84.2
gemini-1.5	88.5	89.5	100.0	90.5	77.5	81.0	90.5	77.0
codestral	88.5	82.9	100.0	88.6	80.0	83.7	91.5	73.7
GPT-4o	94.0	88.3	100.0	90.9	89.0	88.0	91.5	81.1
llama-3	88.0	81.6	100.0	88.5	77.0	79.3	88.0	72.9

TABLE IX
COMPILATION-TIME ERROR TYPES. LLMS: ❶ *CLAUDE-3.5*, ❷ *GEMINI-1.5*, ❸ *CODESTRAL*, ❹ *GPT-4O*, ❺ *LLAMA-3*.

Error Type	Java					C++					C				
	❶	❷	❸	❹	❺	❶	❷	❸	❹	❺	❶	❷	❸	❹	❺
Library Errors	10	17	15	8	17	20	8	4	11	22	7	14	3	3	5
Exception Handling	0	3	2	0	4	-	-	-	-	-	-	-	-	-	-
Class/Member	0	0	4	0	1	3	2	9	3	0	-	-	-	-	-
Syntax	0	1	1	2	0	4	2	6	1	6	-	-	-	-	-
Type Compatibility	0	1	1	0	2	-	-	-	-	-	-	-	-	-	-
Variable def. related	0	1	1	1	1	6	15	5	4	4	-	-	-	-	-
Binding/Qualifier	-	-	-	-	-	0	1	0	0	2	-	-	-	-	-
Undeclared Var./Fun.	-	-	-	-	-	2	13	11	2	6	1	1	4	2	6
Constant Undeclared	-	-	-	-	-	-	-	-	-	-	0	1	2	0	10
Function Argument	-	-	-	-	-	-	-	-	-	-	2	0	0	0	3
Incomplete Code	-	-	-	-	-	-	-	-	-	-	1	1	1	1	0
Other	-	-	-	-	-	-	-	-	-	-	2	3	7	1	0

A. Semantic Soundness

Compilation-Time Errors: The first part of Table VIII presents insights into the performance of different LLMs in generating compilable code without errors in four programming languages (Java, Python, C++, and C). To calculate the compilation success rate, let P be the programming language, C be the number of compilable files in P, T be the total number of files from in P, and S_P be the compilation success rate (in percentage). We then define $C = \sum_{i=1}^{n} C_i$, where C_i is 1 if file i is compileable successfully and 0 otherwise. Moreover, we define $T = n$, where n is the total number of files. We also define the compilation success rate in P as $S_P = (C_P/T_P) \times 100$.

The findings reveal that all models achieve high percentages of compilable code in Python, using three LLMs (*gemini-1.5*, *codestral*, and *GPT-4o*), achieving a success rate 100%. This suggests that Python's simpler syntax and dynamic nature make it easier for the AI tool to generate accurate and error-free code.

> *Takeaway:* Python had the highest success rates for compilable code across all models. This suggests that Python's simpler syntax and dynamic nature contribute to the accuracy and reliability of code generation by AI models.

The variability in compilation-time error-free rates reported in Table VIII for Java, C++, and C indicates that LLMs exhibit different strengths depending on the programming language. For Java, performance fluctuates more than in Python, with success rates ranging from 88.00% to 95.00%. Models such as *claude-3.5* and *GPT-4o* outperform others, while *llama-3*, *gemini-1.5*, and *codestral* show slightly lower rates at 88%, 88.5%, and 88.5%, respectively.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

These results can be attributed to Java's verbose, statically typed nature, which reduces compilation-time errors when an LLM correctly interprets its syntax and principles. The slight performance differences likely reflect how effectively each LLM handles Java's syntax rules, exception handling, and object-oriented features (i.e., encapsulation). Models such as *claude-3.5* and *GPT-4o* appear to be better fine-tuned for Java-specific dependencies and required imports. Missing import statements, such as `java.util.*`, remain a common cause of compilation failures.

> *Takeaway:* Models like *claude-3.5* and *GPT-4o* perform better on Java due to improved handling of syntax rules and package dependencies, while *llama-3*, *gemini-1.5*, and *codestral* show lower success rates, underscoring the importance of Java-specific fine-tuning to improve performance.

C++ shows the lowest success rates across all models, with scores ranging from 77.00% to 89.00%. The worst error-free score in C++ is with *llama-3* and *gemini-1.5*. The performance of LLMs in generating C++ code appears to be constrained by missing include statements, incorrect type handling, and misinterpretation of APIs. C++ is a complex language with many different standards (i.e., C++11, C++17), and LLMs struggle when dealing with the complexities of the language's syntax, library usage, and type system, leading to frequent compile-time errors when generating more advanced code involving libraries like STL (Standard Template Library) or 3 rd party APIs/libraries.

> *Takeaway:* LLMs struggle with C++ due to missing includes, type errors, standard mismatches (i.e., C++11 vs. C++17), and API misinterpretations. The language's syntax complexity and strict type system lead to frequent compile-time errors, especially with STL and third-party libraries.

The varying success rates in C code generation reflect differences in handling C standards and dependencies. The high success rate of *claude-3.5* (96%) indicates adherence to C conventions, including proper header inclusion and type declarations. In contrast, *llama-3*, with the lowest success rate (88.00%), frequently produced errors involving undeclared types (i.e., `bool`), missing headers (e.g., `cgi/cgi.h`), and incorrect function arguments. These issues suggest reliance on non-standard libraries or flawed assumptions about the compilation environment. Such inconsistencies highlight the need to generate code that follows standard C practices, includes necessary dependencies, and respects function signature conventions to improve compilation.

> *Takeaway:* The frequent errors in C are related to unknown types (i.e., bool), missing headers (i.e., `cgi/cgi.h`), and incorrect function arguments, are due to reliance on non-standard libraries. Ensuring generated code aligns with standard C practices and includes all dependencies with its source is crucial for successful compilation.

The percentage of code files without compilation errors provides insight into how well these AI tools are tuned for different programming paradigms and syntax complexities. Regarding *claude-3.5*, it performs well in Java with 95% as a result, and in C with 96%, showing its good performance with both object-oriented and functional programming languages. Less performance with the C++ programming language with 81.5%. *GPT-4o* achieved strong results in all program languages, especially C++ (89%), showing that it is reliable and adaptable for generating error-free code. Regarding *gemini-1.5*, this model achieved the highest performance in Python and good in C, but this model achieved the lowest performance as results regarding Java and C++ compared to *claude-3.5* and *GPT-4o*. Lastly, regarding *llama-3*, performs lower in most languages except Python. It struggles with Java (88%), C++ (77%), and C (88%), showing that it is not as well tuned for more complex languages. The reason behind the results shown for each of the AI tools refers to a set of factors such as training data, since each model trained on large, more diverse, and higher-quality datasets is better at understanding various programming languages [51]. Another important factor regarding the performance of the AI tool refers to model size and complexity, since a larger model with more parameters such as *GPT-4o* can better understand and generate.

Types of Compilation-Time Errors: A breakdown of the compilation-time errors is summarized in Table IX. Python's breakdown is excluded from the table since it lists only error types occurring more than once across all LLMs. In the following, compilation errors are grouped into categories to understand the common failure modes across the models.

① *Library Errors:* The most common error category across all models involved missing imports or incorrect package references. Frequent issues included "cannot find symbol" errors and failures to recognize or include required third-party libraries, particularly missing imports such as `java.util.*`, especially in tasks involving non-standard libraries. These patterns suggest a need for better handling of Java libraries and packages, potentially through training LLMs on more diverse codebases with correct import usage. In addition, improving the contextual understanding of external dependencies may enhance model performance in such tasks.

② *Exception Handling:* Missing or incorrect exception handling was another area where the models faltered. *gemini-1.5*, *codestral*, and *llama-3* were particularly prone to this type of error, indicating that these models could improve by focusing on the need to handle exceptions based on the exception that can be triggered by code in scope for a particular code block or method.

③ *Missing Class/Member:* An often recurrent issue flagged by *codestral* is the absence of a class or class member, such as setter and/or getter methods. This error arises when the model presumes the presence of a specific method or class without including the pertinent code or even suggesting it within comments.

④ *Syntax:* Syntax errors, including incorrect symbols, missing semicolons, or invalid syntax, were found primarily

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

in *codestral*. This pattern suggests a need for improved fine-tuning on Java syntax conventions and a deeper understanding of language-specific rules.

⑤ *Type Compatibility:* Errors related to incompatible types, such as attempts to assign a java.lang.Object to a java.lang.String, or java.io.IOException cannot be converted to java.lang.SecurityException. These errors indicate the models' ability to infer the data types in context.

⑥ *Variable Identifier:* Errors arising from the use of undefined variables were found in multiple models. This indicates that the models occasionally fail to maintain variable state consistency across different sections of the code.

⑦ *Undeclared Var./Fun:* Errors where a variable or function is used before being declared, leading to a failure in recognizing identifiers. This may happen if a necessary declaration or definition is missing.

⑧ *Binding/Qualifier:* Errors related to failing to bind a variable correctly or mismatches with qualifiers. These errors often occur in function parameters or references.

⑨ *Constant Undeclared:* Errors with undeclared constants like `true` and `nullptr`.

Semantic Correctness Using Unit Test: The results in Table VIII summarize semantic correctness across languages. Most models exhibit consistent performance, though accuracy varies with language complexity. *claude-3.5* achieved up to 92.1%, while *GPT-4o* ranged from 90.9% (Python) to 81.1% (C). *gemini-1.5* showed a broader spread, from 90.5% to 77.0%, and *codestral* ranged from 88.6% to 73.7%. *llama-3* yielded the lowest scores overall, peaking at 88.5% in Python. These differences underscore the sensitivity of LLM outputs to both prompt formulation and target language. Although tasks require similar reasoning, performance discrepancies suggest underlying differences in how models interpret and implement logic, likely influenced by training data, architecture, or optimization strategies. The wider variability seen in *gemini-1.5*, *llama-3*, and *codestral* may reflect challenges in handling specific language features or complex constructs, pointing to design or training gaps.

Takeaway: The variance in success rates between different LLMs, with an average of 14.7% for *gemini-1.5*, *llama-3*, and *codestral*, indicates notable differences in how these models handle coding tasks, likely due to variations in training data, model design, and optimization strategies.

For Java, the models generally showed strong performance, with *claude-3.5* achieving the highest success rate of 91.8%. This indicates a good proficiency in creating effective and precise Java code, probably because of Java's object-oriented design and consistent libraries, which apparently correspond well with the models' training data. *gemini-1.5* also performed well, reaching a success rate of 89.5%, demonstrating its proficiency in managing Java syntax and standard programming constructs. *codestral* and *llama-3* showed slightly lower success rates at 82.9% and 81.6%, respectively. The variation in performance, especially for *codestral* and *llama-3*, could indicate difficulties in addressing Java's more complex class structures or standard library usage, which aligns with its relatively lower compilation success rate. On the whole, the high semantic correctness across models confirms their capacity to generate functional Java code, while also highlighting areas where deeper reasoning or structural precision remains necessary.

Takeaway: LLMs perform strongly on Java, with *claude-3.5* achieving the highest score at 91.8%, likely reflecting Java's structured object-oriented design and stable libraries. *gemini-1.5* follows at 89.5%, while *codestral* and *llama-3* score lower at 82.9% and 81.6%, respectively. These results indicate overall proficiency but also expose remaining challenges in handling Java-specific complexity.

Among the evaluated languages, Python achieved the highest success rates, with *claude-3.5* leading at 92.1%, followed by *GPT-4o* and *gemini-1.5* at 90.9% and 90.5%, respectively. Python's simplicity and dynamic typing likely contribute to these high success rates, as its syntax is generally more permissive and flexible than statically typed languages. The consistently strong performance across models suggests they are well-versed in Python constructs. Notably, *llama-3* achieved a slightly lower success rate of 88.5%, indicating that even high-performing models may encounter challenges in generating correct Python code, potentially due to differences in language handling or reasoning strategies. For C++, *claude-3.5* achieved the highest semantic success rate at 91.2%, demonstrating a strong capability in handling C++ syntax and object-oriented constructs. *GPT-4o* followed at 88.0%, showing reliable performance on complex language features. *codestral* reached 83.7%, while *gemini-1.5* and *llama-3* trailed at 81.0% and 79.3%, respectively, indicating more pronounced challenges in generating correct C++ code across these models. Although these trends, such as Python demonstrating fewer compilation errors than C++, align with practitioners' expectations, our results quantify the magnitude of these differences across models and link them to specific language quality and security outcomes.

Takeaway: claude-3.5 achieves the highest success rate in C++ at 91.2%, then *GPT-4o* at 88.0% and *codestral* at 83.7%, indicating differences in handling C++ complexity. C remains more challenging across models due to stricter syntax, with *claude-3.5* leading at 84.2% and *llama-3* scoring lowest at 72.9%. These results underscore ongoing limitations in low-level language reasoning and generation.

Success rates in C were notably lower across all models, with *claude-3.5* achieving the highest at 84.2%, followed by *GPT-4o* at 81.1%, *gemini-1.5* at 77.0%, *codestral* at 73.7%, and *llama-3* at 72.9%. This performance gap can be attributed to C's stricter requirements, including manual memory management, explicit type declarations, and the mandatory inclusion of standard headers. These findings underscore the need for targeted enhancements in LLM-based code generation to better address

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE X
STATIC FEATURES: LINES OF CODE (LOC), CYCLOMATIC COMPLEXITY (CYC), COMPLEXITY DENSITY (CCD), AND COGNITIVE COMPLEXITY (COC)

	Java				Python				C++				C			
Model	LoC	CyC	CCD	CoC	LoC	CyC	CCD	CoC	LoC	CyC	CCD	CoC	LoC	CyC	CCD	CoC
claude-3.5	6,480	1,097	0.17	891	3,474	895	0.26	881	6,725	1,261	0.19	1,096	7,509	1,464	0.19	1,463
gemini-1.5	5,606	965	0.23	902	3,064	765	0.37	881	5,822	1,084	0.25	1,044	6,555	1,256	0.28	1,331
codestral	4,143	730	0.13	585	2,077	553	0.18	491	4,293	825	0.14	732	4,532	894	0.14	869
GPT-4o	5,328	908	0.17	670	2,906	665	0.23	610	5,263	676	0.13	510	5,926	1,133	0.19	994
llama-3	4,993	913	0.18	809	2,633	677	0.26	631	5,077	979	0.19	884	5,006	1,004	0.20	1,012
Summary	**26,550**	**4,613**	**0.17**	**3,857**	**14,154**	**3,555**	**0.25**	**3,494**	**27,180**	**4,825**	**0.18**	**4,266**	**29,528**	**5,751**	**0.19**	**5,669**

the syntactic rigor and low-level control demanded by the C programming language.

Takeaway: Semantic correctness is high in Python and Java for top-performing models, but drops in C++ and especially C, where success rates range roughly from 73% to 84%. This gap shows that the benchmark is not uniformly easy and exposes language-dependent challenges, particularly in lower-level languages with manual memory management and stricter type systems.

B. Complexity

This section evaluates the static code features of LLMs generated code across Java, Python, C++, and C using several complexity metrics, such as lines of code (LoC), cyclic complexity (CyC), cyclic complexity density (CCD) and cognitive complexity (CoC). The analysis included code generated by various LLMs identified as *claude-3.5*, *gemini-1.5*, *codestral*, *GPT-4o*, and *llama-3*. The results are provided in Table X. For Java code, as summarized in Table X, shows that *claude-3.5* produced the highest LoC (6,480) and the highest CoC (891). *gemini-1.5* showed the highest CCD (0.23), indicating higher complexity. Conversely, *codestral* generated relatively less complex code with the lowest CyC (730) and CoC (585). The evaluation of the Python code showed that among the models, *gemini-1.5* had the highest CCD (0.37), reflecting a very dense complexity despite having a lower LoC (2,077). *claude-3.5* produced the highest LoC (3,474) and the highest CoC (881) the same as *gemini-1.5*. *codestral* presented the lowest values in all metrics, indicating simpler and less complex code.

For the C++ code, the results indicate that *claude-3.5* generated the highest LoC (6,725) and the highest CoC (1,096). The highest CCD was observed with *gemini-1.5* (0.25), suggesting more intricate code patterns. In contrast, *GPT-4o* showed the lowest CyC (676) and CoC (510), indicating simpler, more maintainable code.

With respect to the C code, *claude-3.5* again led with the highest LoC (7,509) and CoC (1,463). *gemini-1.5* had the highest CCD (0.28), while *codestral* presented with the least CyC (894) and CoC (869), indicating a simpler code.

Based on the analysis above, the CCD varies between languages, reflecting differences in code density and distribution. CoC closely follows CyC but offers additional insight into code readability and maintainability. *claude-3.5* generally produces more verbose and complex code across all languages, suggesting greater difficulty in readability and even the runtime overhead. Lastly, *codestral* and *GPT-4o* tend to generate simpler and more maintainable code, with lower complexity metrics.

C. Security and Quality Attribute

The security and quality evaluation is based on the different metrics in Section III.

Table XI presents Java results across key quality attributes. For lines of code (LoC), *claude-3.5* generates the highest output, contributing to increased complexity and variability. In contrast, *codestral* is the most efficient model in terms of LoC. Regarding security, which assesses resistance to unauthorized access and vulnerabilities, *gemini-1.5* yields the highest number of vulnerabilities among the LLMs for Java. This outcome is likely due to a relative lack of security-focused training data compared to other models.

Per Table XI, *codestral* has the fewest reliability issues (37), indicating minimal crashes or errors, likely due to its lower lines of code compared to other models. It also ranks best in maintainability, with 535 issues, reflecting code that is easier to understand, modify, and extend, attributable to clear structure, comments, and function decomposition. For security hotspots, *llama-3* performs best, with only 27 issues.

For the Python code evaluation in Table XI, the models show notable variations in quality. *claude-3.5*, generating 3,474 lines, has 82 maintainability issues and 44 security hotspots, indicating areas needing security improvements. *codestral*, with 2,077 lines, performs better with 50 maintainability issues and 39 security hotspots, reflecting better overall control. *gemini-1.5* and *GPT-4o*, producing 3,064 and 2,906 lines respectively, show similar outcomes with maintainability issues (83 and 75) and security hotspots (36 and 43). *llama-3*, generating 2,633 lines, has the fewest security hotspots (34) and moderate maintainability issues (63), indicating balanced but not flawless performance.

Table XI reveals that *claude-3.5*, with 6,725 LoC, has the highest maintainability issues (590) but few security hotspots (36), suggesting strong security but complex maintenance. *codestral*, generating 4,293 LoC, has the fewest maintainability issues (470) and security hotspots (20), indicating a balanced performance. *gemini-1.5*, producing 5,822 LoC and with a higher count of both security issues (17) and maintainability concerns (587). *GPT-4o* and *llama-3*, with 5,263 and 5,077 lines, respectively, demonstrate moderate and balanced performance in both areas.

For C, Table XI shows that *claude-3.5*, with 7,509 LoC, has a high number of maintainability issues (449) and security hotspots (186), indicating difficulties in managing large and

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
QUALITY ATTRIBUTES AGAINST SECURITY (S), RELIABILITY (R), MAINTAINABILITY (M), AND SECURITY HOTSPOTS (SH)

	Java				Python				C++				C			
Model	S	R	M	SH	S	R	M	SH	S	R	M	SH	S	R	M	SH
claude-3.5	15	51	810	61	5	0	82	44	9	6	590	36	19	60	449	186
gemini-1.5	16	46	694	33	9	1	83	36	17	8	587	18	33	50	352	146
codestral	12	37	535	40	9	1	50	39	10	5	470	20	21	39	314	115
GPT-4o	13	52	925	76	8	0	75	43	7	6	459	17	27	37	380	133
llama-3	13	46	932	27	9	1	63	34	10	10	565	18	22	54	322	182
Summary	**69**	**232**	**3896**	**237**	**40**	**3**	**353**	**196**	**53**	**35**	**2671**	**109**	**122**	**240**	**1817**	**762**

TABLE XII
CWE DOMAIN-LEVEL BREAKDOWN OF DETECTED VULNERABILITIES ACROSS PROGRAMMING LANGUAGES AND LLMS. LLMS: ❶ CLAUDE-3.5, ❷ GEMINI-1.5, ❸ CODESTRAL, ❹ GPT-4O, ❺ LLAMA-3.

	Java					Python					C++					C				
CWE Domain	❶	❷	❸	❹	❺	❶	❷	❸	❹	❺	❶	❷	❸	❹	❺	❶	❷	❸	❹	❺
Input Validation	6	8	5	7	6	3	5	5	3	3	0	7	3	1	0	0	6	3	3	0
Injection	3	4	3	2	3	0	1	1	1	2	0	2	1	1	2	1	2	1	2	3
Buffer Errors	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	19	14	13	11
Access Control	2	2	2	1	4	2	2	2	2	3	-	-	-	-	-	-	-	-	-	-
Cryptographic Issues	4	0	0	0	0	0	0	0	0	0	9	8	6	5	7	9	5	3	8	7
Information Exposure	2	2	2	2	1	0	1	0	0	0	0	0	0	0	1	-	-	-	-	-
Resource / Memory Errors	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	1	0	1	1

secure code. *codestral*, producing 4,532 lines, has fewer security hotspots (115) and maintainability issues (314), though it scores lower in reliability (39). *gemini-1.5*, with 6,555 lines, shows the most security issues (33) and moderate maintainability concerns (352), signaling significant security problems. *GPT-4o*, generating 5,926 lines, balances moderate security hotspots (133) and maintainability issues (380), while *llama-3*, with 5,006 lines, shows higher reliability (54) but faces substantial security hotspots (182).

To illustrate our findings, Appendix C, available online presents examples of reliability, maintainability, and security hotspot issues in LLM-generated code. These cases show how seemingly minor flaws such as missing input validation, outdated API usage, or unsafe regular expressions can compromise code quality, even in relatively simple tasks.

CWE Categories of Security Quality Attribute: The detection of security flaws in the code produced by five Large Language Models (LLMs) for Java, Python, C, and C++ highlights substantial differences in both the quality and security of the code among the models and programming languages. The Table XII presents a breakdown of identified CWE categories, reflecting the different LLMs strengths and weaknesses in secure code generation. This analysis sheds light on common security challenges that various LLMs might present when producing code, identifying opportunities to enhance LLM-driven code generation.

In Java, the most observed CWE is CWE-780 (Use of RSA Algorithm without OAEP), which remains consistently high in frequency across various models, notably with models *claude-3.5*, *gemini-1.5*, and *GPT-4o*. This suggests that while generating code for encryption tasks, many LLMs fail to apply the necessary secure padding scheme (OAEP), which is essential for RSA encryption security. This could lead to insecure cryptographic implementations if used in real-world applications. Additionally, CWE-259 (Use of Hard-coded Password) and CWE-295 (Improper Certificate Validation) appear frequently, suggesting that certain models tend to generate credentials or handle certificates in an insecure manner. Issues such as CWE-611 (Improper Restriction of XML External Entity Reference) indicate a common oversight in XML handling, potentially exposing generated code to XML external entity (XXE) attacks.

In Python, CWE-780 is also frequently observed, though less than in Java, indicating that RSA padding issues are not exclusive to Java but persist across languages. Another notable vulnerability in Python is CWE-259, which highlights the common use of hard-coded credentials, posing a risk to sensitive information if deployed. Moreover, Python's handling of CWE-79 (Improper Neutralization of Input During Web Page Generation) reveals weaknesses in web-based output, leading to cross-site scripting (XSS) vulnerabilities.

C++ exhibits a range of vulnerabilities, with CWE-295, CWE-326, and CWE-327 (related to improper certificate validation, inadequate encryption strength, and the use of a broken or risky cryptographic algorithm, respectively) being highly prevalent. These vulnerabilities suggest that cryptographic practices and certificate handling in C++ code generated by LLMs are notably insecure. Additionally, CWE-780 (RSA without OAEP) and CWE-611 (XXE) are also present, indicating potential security weaknesses in cryptographic and XML-handling implementations. This underscores that certain models do not enforce secure practices when generating security-sensitive code in C++. The widespread presence of CWE-297 (Improper Validation of Certificate with Host Mismatch) further highlights weaknesses in certificate validation, which could expose systems to man-in-the-middle (MITM) attack vectors if deployed.

For C, there is a high prevalence of CWE-120 (Buffer Copy without Checking Size of Input), reflecting common buffer

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
EVALUATION OF CLEAN CODE IN TERMS OF CONSISTENCY (C), INTENTIONALITY (I), ADAPTABILITY (A), AND RESPONSIBILITY (R)

	Java				Python				C++				C			
Model	C	I	A	R	C	I	A	R	C	I	A	R	C	I	A	R
claude-3.5	0	203	532	10	54	23	5	5	116	438	42	9	162	321	36	9
gemini-1.5	0	244	387	9	47	34	5	7	136	421	40	15	188	197	38	11
codestral	0	181	182	6	35	17	1	7	117	335	24	9	155	186	27	6
GPT-4o	8	148	392	9	47	27	4	5	93	334	39	6	197	210	25	11
llama-3	61	200	360	9	51	15	1	6	121	408	49	7	160	205	25	7
Summary	**69**	**976**	**1,853**	**43**	**234**	**116**	**16**	**30**	**583**	**1,936**	**194**	**46**	**862**	**1,119**	**151**	**44**

overflow vulnerabilities in low-level languages that require manual memory management. The frequent occurrences of CWE-295, CWE-326, and CWE-327 indicate that LLMs often generate C code with poor cryptographic practices and inadequate certificate validation, a trend also observed in C++. Additionally, CWE-131 (Incorrect Calculation of Buffer Size) and CWE-788 (Access of Memory Location After End of Buffer) highlight inadequate buffer management, posing serious security risks such as memory corruption and arbitrary code execution. CWE-780 appears frequently in *gemini-1.5*, *codestral*, and *GPT-4o*, indicating RSA padding issues, though at lower rates than in higher-level languages like Java and Python.

In summary, there is considerable variation in the security quality of the generated code across different programming languages. *gemini-1.5* exhibited the highest overall reported vulnerabilities, suggesting less conservative or secure default behaviors. However, some vulnerabilities persist across all models and languages.

Vulnerabilities in languages like C and C++ are more skewed towards memory management issues (i.e., buffer overflows, incorrect buffer size calculations) compared to Java and Python, where cryptographic and XML-related vulnerabilities are more common. This highlights inherent language-specific risks, such as memory safety in C and C++ and secure API use in Java and Python.

To illustrate identified security vulnerabilities, Appendix C, available online presents two representative cases: RSA encryption without OAEP padding (CWE-780) and hard-coded passwords (CWE-259), corresponding to tasks 106 and 83. These examples show that LLMs may default to insecure practices when safer alternatives exist, indicating limited contextual security awareness. These CWEs are well known and documented in prior analyses of LLM-generated and human-written code. The goal here is to quantify how often such known weaknesses, along with other vulnerabilities, appear across recent LLMs and programming languages under a unified benchmark. The patterns in Table XII therefore support comparative analysis of model and language behavior rather than claims of novel vulnerability types.

D. Clean Code

Table XIII shows the clean code analysis for Java, where most models handled consistency well, with no issues reported except for *GPT-4o* (8) and *llama-3* (61). However, intentionality issues were prominent, especially for *claude-3.5* (203) and *gemini-1.5* (244), suggesting that the clarity of code purpose could be improved. Adaptability scores were highest for *claude-3.5* (532) and lowest for *codestral* (182), indicating varying levels of code flexibility for future changes.

In the same table, the clean code analysis for Python shows slightly more noticeable consistency issues, especially for *claude-3.5* (54) and *llama-3* (51), while the intentionality attribute remains a common challenge across all models. Adaptability and responsibility issues were low across the board, highlighting Python's inherent simplicity and flexibility. This suggests that while Python code is generally adaptable, models need to improve its clarity.

With regard to C++, Table XIII also shows significant challenges, particularly with intentionality. *claude-3.5* (438) and *gemini-1.5* (421) had the highest intentionality issues, reflecting difficulties in generating code that clearly communicates its purpose. Consistency was also a challenge for all models, with no significant peaks in reducing the problems. In addition, for the C language, we can see that intentionality and consistency issues are prevalent, with *GPT-4o* (197) and *gemini-1.5* (188) ranking highest in consistency issues. All models exhibited relatively high adaptability issues, particularly *claude-3.5* (36) and *gemini-1.5* (38), reflecting C's complexity in managing clear and adaptable code. These results show that C presents challenges in clean code generation in all models.

E. Comparison With Related Work

Our findings align with and extend several recent studies on LLM-based code generation. For example, Codex was shown [52] to perform well in Python tasks, but with reduced accuracy in lower-level languages such as C, a pattern that we also observed across multiple models. Similarly, CodeT5 was shown to produce strong results in code understanding and generation [53], but its evaluation mainly focused on syntactic correctness, without assessing deeper aspects such as security or maintainability.

Siddiq et al. [22] introduced SecurityEval, a Python-only dataset containing 130 prompts and corresponding LLM-generated code snippets labeled with CWE vulnerability types. Their setup includes predefined function signatures that constrain input/output behavior, which simplifies the generation task. While their dataset is useful for structured vulnerability evaluation, it targets only a single model and does not consider functional correctness or code maintainability. In contrast, our study uses natural, unconstrained prompts, more reflective of real developer queries, and evaluates multiple LLMs across four languages. We combine correctness testing, maintainability

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

scoring, and security assessment, offering a more comprehensive and realistic perspective on LLM behavior.

Lenarduzzi et al. [25] compared the detection performance and agreement of six static analysis tools, highlighting inconsistency and the need for manual validation. Their findings support our approach of manual validation on the analysis results of approximately 20% of the tasks to ensure an accurate interpretation of security warnings. Tony et al. [24] presented LLMSecEval, a dataset of 150 natural language prompts targeting 18 high-impact CWEs, along with secure code examples for two languages. Although their prompts are language-agnostic and geared toward security benchmarking, their study focuses on secure-only outputs and a limited language set. Our work differs by evaluating unconstrained LLM output, assessing not only security, but also correctness and maintainability across four languages and five models. This enables a more holistic evaluation grounded in realistic developer-facing scenarios.

What sets our work apart is its focus on model and language diversity, developer-realistic prompting, and multidimensional quality analysis. Prior studies, such as [27], demonstrated that GitHub Copilot often produces insecure code by analyzing 89 scenarios with CWE vulnerabilities. However, their evaluation was limited to a single model. In contrast, we assess five LLMs using CWE-based tagging, static analysis, and unit test validation. This enables a comprehensive evaluation across multiple dimensions, syntax, correctness, complexity, reliability, and security. Our findings both reaffirm previously reported limitations and offer new insights into when and why LLMs struggle to generate high-quality code across varied programming contexts.

VI. Threats to Validity

We consider several validity threats.

Construct Validity: We evaluate quality and security using SonarQube and CodeQL, which provide scalable rule-based analysis but may produce false positives and lack full semantic reasoning. To mitigate this, we manually validated about 20% of tasks via stratified sampling. Semantic correctness is measured through unit tests, which may miss untested behaviors if coverage is incomplete.

Internal Validity: We used non-deterministic decoding (temperature $= 0.9$) to reflect realistic developer usage. To assess sensitivity, we conducted a supplementary ablation with a lower temperature setting; model rankings and cross-language trends remained stable, indicating robustness to reasonable parameter variation. Identical prompts were used across models and languages to ensure fairness. Function signatures were omitted to mirror real-world usage, requiring task-specific test adaptation. Consistency was enforced through shared test specifications and cross-review validation. All experiments used fixed model versions.

External Validity: The benchmark comprises 200 tasks across four languages and five models, covering common scenarios with self-contained, single-file programs and does not capture large-scale systems or complex dependencies. Accordingly, security findings reflect code-level properties (e.g., cryptographic misuse, hard-coded secrets, missing validation) rather than end-to-end system security. Interactive or IDE-assisted workflows are not evaluated.

Temporal Validity and Reproducibility: LLMs evolve rapidly; absolute performance values and rankings may change over time. However, observed cross-language patterns and recurring vulnerability categories are likely to persist. The results represent a time-stamped snapshot of evaluated model behavior. To support replication and extension, we release the dataset, prompts, unit tests, and analysis scripts within a model-agnostic evaluation framework.

Conclusion Validity: We applied standardized prompts, evaluation criteria, and reporting metrics across all models and languages. Reproducibility may be affected by future model updates or API changes. The study evaluates single-turn generation and does not address multi-turn or collaborative development settings.

VII. Conclusion, Limitations and Future Work

Code quality and security vary substantially across programming languages and models. Python and Java exhibit higher semantic success, while C and C++ show greater compilation and memory-safety challenges. Although several models achieve moderate functional correctness, security hotspots remain common, and modern language safeguards are often underutilized. These findings indicate that semantic correctness alone does not ensure security or robustness. Performance differences reflect the influence of language abstractions, structural complexity, and model training characteristics. The results provide a time-stamped baseline of current LLM capabilities and establish a reproducible reference point for tracking future progress.

Future Work: Future work will examine the impact of prompt engineering and multi-turn interactions on code quality and security. Moreover, extending evaluation beyond single-file tasks to multi-module systems with real dependencies will enable analysis of system-level properties such as authentication flows, state management, and cross-component data handling. We also plan to incorporate formal coverage metrics, additional static analyzers, and dynamic techniques such as fuzz testing to strengthen vulnerability detection and robustness evaluation. The current benchmark serves as a controlled baseline for these richer and more realistic settings.

References

[1] Microsoft, "Intellicode - visual studio marketplace," 2024, Accessed: Dec. 05, 2024. [Online]. Available: https://marketplace.visualstudio.com/items?itemName=VisualStudioExptTeam.vscodeintellicode

[2] GitHub, "Introducing GitHub copilot: Your AI pair programmer - the github blog," 2024, Accessed: Dec. 0, 2024. [Online]. Available: https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer/

[3] O. Asare, M. Nagappan, and N. Asokan, "A user-centered security evaluation of copilot," in *Proc. Int. Conf. Softw. Eng.*, 2024, Art. no. 158.

[4] R. Elgedawy et al., "Ocassionally secure: A comparative analysis of code generation assistants," 2024, *arXiv:2402.00689*.

[5] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2023, pp. 2785–2799.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.

[6] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by chatGPT," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2023, pp. 2445–2451.

[7] M. L. Siddiq and J. C. S. Santos, "Generate and pray: Using SALLMS to evaluate the security of LLM generated code," 2023, *arXiv:2311.00889*.

[8] M. Nair, R. Sadhukhan, and D. Mukhopadhyay, "Generating secure hardware using chatGPT resistant to cwes," *IACR Cryptol. ePrint Arch.*, p. 212, 2023. [Online]. Available: https://eprint.iacr.org/2023/212

[9] G. Ras, N. Xie, M. van Gerven, and D. Doran, "Explainable deep learning: A field guide for the uninitiated," *J. Artif. Intell. Res.*, vol. 73, pp. 329–396, 2022, doi: 10.1613/jair.1.13200.

[10] G. Ras, N. Xie, M. van Gerven, and D. Doran, "ppdb1123/copilot-user-study-supp," 2024, Accessed: May 14, 05, 2024. [Online]. Available: https://github.com/ppdb1123/copilot-user-study-supp

[11] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at C: A user study on the security implications of large language model code assistants," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 2205–2222.

[12] O. Asare, M. Nagappan, and N. Asokan, "Is github's copilot as bad as humans at introducing vulnerabilities in code?," *Empir. Softw. Eng.*, vol. 28, no. 6, 2023, Art. no. 129, doi: 10.1007/s10664-023-10380-1.

[13] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 508–512, doi: 10.1145/3379597.3387501.

[14] B. Yetistiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of AI-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatGPT," 2023, *arXiv:2304.10778*.

[15] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[16] M. Chen et al., "Raphaelkhoury/programsgeneratedbychatGPT: Programs generated by chatGPT," 2024, Accessed: May 14, 2024. [Online]. Available: https://github.com/RaphaelKhoury/ProgramsGeneratedByChatGPT

[17] F. Wu et al., "Exploring the limits of chatGPT in software security applications," 2023, *arXiv:2312.05275*.

[18] P. Black, "Juliet 1.3 test suite: Changes from 1.2," Gaithersburg, MD, USA: U.S. Department of Commerce, National Institute of Standards and Technology, Jun. 2018.

[19] P. Black, "tuhh-softsec/LLMSecEval," 2024, Accessed: May 14, 2024. [Online]. Available: https://github.com/tuhh-softsec/LLMSecEval,

[20] R. Schuster, C. Song, E. Tromer, and V. Shmatikov, "You autocomplete me: Poisoning vulnerabilities in neural code completion," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1559–1575.

[21] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 862–880.

[22] M. L. Siddiq and J. C. S. Santos, "Securityeval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proc. Int. Workshop Mining Softw. Repositories Appl. Privacy Secur.*, 2022, pp. 29–33.

[23] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," in *Proc. IEEE Conf. Softw. Testing Verification Validation*, 2025, pp. 103–114.

[24] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, "LLMseceval: A dataset of natural language prompts for security evaluations," in *Proc. 20th IEEE/ACM Int. Conf. Mining Softw. Repositories*, Melbourne, Australia, 2023, pp. 588–592, doi: 10.1109/MSR59073.2023.00084.

[25] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *J. Syst. Softw.*, vol. 198, 2023, Art. no. 111575, doi: 10.1016/j.jss.2022.111575.

[26] Y. Huang et al., "Generative software engineering," 2024, *arXiv:2403.02583*.

[27] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of github copilot's code contributions," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 754–768.

[28] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Code quality, security & static analysis tool with SonarQube," 2024, Accessed: May 05, 2024. [Online]. Available: https://www.sonarsource.com/products/sonarqube/

[29] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "GPT-4| OpenAI," 2024, Accessed: May 12, 2024. [Online]. Available: https://openai.com/index/gpt-4/

[30] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Perplexity," 2024, Accessed: Sep. 05, 2024. [Online]. Available: https://www.perplexity.ai/

[31] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Claude," 2024, Accessed: Sep. 05, 2024. [Online]. Available: https://claude.ai/new

[32] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Mistral AI| frontier AI in your hands," 2024, Accessed: May 12, 2024. [Online]. Available: https://mistral.ai/

[33] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Gemini - chat to supercharge your ideas," 2024, Accessed: May 12, 2024. [Online]. Available: https://gemini.google.com/

[34] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Most popular programming languages in 2024 & beyond," 2024, Accessed: May 12, 2024. [Online]. Available: https://www.orientsoftware.com/blog/most-popular-programming-languages/

[35] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "C programming—The state of developer ecosystem in 2023 infographic. JetBrains: Developer tools for professionals and teams," 2024, Accessed: Sep. 07, 2024. [Online]. Available: https://www.jetbrains.com/lp/devecosystem-2023/cpp/

[36] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "CodeQL," 2025, Accessed: Dec. 05, 2025. [Online]. Available: https://codeql.github.com/

[37] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "JaCoCo/jacoco: Microscope: Java code coverage library," 2025, Accessed: Jul. 09 2025. [Online]. Available: https://github.com/jacoco/jacoco

[38] A. H. Watson, D. R. Wallace, and T. J. McCabe, in *Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric*, vol. 500, no. 235, Gaithersburg, MD, USA: U.S. Department of Commerce, Technology Administration, National Institute of Standards and Technology, 1996.

[39] N. E. Fenton and M. Neil, "A critique of software defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 25, no. 5, pp. 675–689, 1999, doi: 10.1109/32.815326.

[40] M. M. Barón, M. Wyrich, and S. Wagner, "An empirical validation of cognitive complexity as a measure of source code understandability," in *Proc. ACM / IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2020, pp. 5:1–5:12, doi: 10.1145/3382494.3410636.

[41] M. M. Barón, M. Wyrich, and S. Wagner, "Problems - leetcode," 2024, Accessed: Sep. 09, 2024. [Online]. Available: https://leetcode.com/problemset/

[42] M. M. Barón, M. Wyrich, and S. Wagner, "Edabit // learn to code with 10,000 interactive challenges," 2024, Accessed: Sep. 09, 2024. [Online]. Available: https://edabit.com/,

[43] M. M. Barón, M. Wyrich, and S. Wagner, "Codewars - achieve mastery through coding practice and developer mentorship," 2024, Accessed: Sep. 09, 2024. [Online]. Available: https://www.codewars.com/

[44] M. M. Barón, M. Wyrich, and S. Wagner, "CWE list version 4," 16," 2024, Accessed: Dec. 16, 2024. [Online]. Available: https://cwe.mitre.org/data/index.html

[45] L. Wang et al., "Performance review on LLM for solving leetcode problems," 2025, *arXiv:2502.15770*.

[46] M. Merkel and J. Dörpinghaus, "A case study on the transformative potential of AI in software engineering on leetcode and chatGPT," 2025, *arXiv:2501.03639*.

[47] Y. Xia et al., "LeetCodeDataset: A temporal dataset for robust evaluation and efficient training of code LLMs," 2025, *arXiv:2504.14655*.

[48] N. Raihan et al., "On the performance of large language models on introductory programming assignments," *J. Intell. Inf. Syst.*, 2025, doi 10.1007/s10844-025-00968-y.

[49] N. Raihan et al., "Exercism," 2025, Accessed: Jul. 4, 2025. [Online]. Available: https://exercism.org/

[50] N. Raihan et al., "CodeQL CWE coverage," 2025, Accessed: Jul. 4, 2025. [Online]. Available: https://codeql.github.com/codeql-query-help/codeql-cwe-coverage/

[51] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 159:1–159:25.

[52] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[53] Y. Wang, W. Wang, S. Joty, and S.-K. Ng, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 8696–8708.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 20,2026 at 10:42:57 UTC from IEEE Xplore. Restrictions apply.